OpenClaw: do zero ao sistema de agentes

Guia passo a passo. Tudo que fizemos desde a instalação até ter 10 agentes operando, com segurança, memória, crons e automações.

ISIS MOREIRA | MARÇO 2026 | GUIA PRÁTICO

Contratar e preparar o *servidor*

01

Tudo começa com um VPS. Sem servidor, não tem agente 24/7.

1. **Contratar um VPS**
   Hostinger, Hetzner ou DigitalOcean. Recomendação mínima: 2 vCPU, 4GB RAM, 80GB SSD. Custo: ~R$25/mês. Nós usamos Hostinger KVM4 (4 vCPU, 16GB RAM, 200GB NVMe).

2. **Acessar via SSH**
   Com o IP do servidor e a senha root, conectar via terminal.

   ```
   ssh root@seu-ip-do-servidor
   ```

3. **Criar usuário dedicado**
   Nunca rodar o agente como root. Criar usuário próprio com home directory.

   ```
   adduser openclaw && usermod -aG sudo openclaw
   ```

4. **Instalar Node.js (v22+)**
   OpenClaw roda em Node.js. Instalar via nvm ou apt.

   ```
   curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash - && sudo apt install -y nodejs
   ```

5. **Instalar OpenClaw**
   Instalação global via npm. Uma linha.

   ```
   npm i -g openclaw
   ```

6. **Rodar o onboard**
   Assistente interativo que configura tudo: modelo, canal, identidade.

   ```
   openclaw onboard
   ```

Atenção: Se você vai usar Telegram, tenha o token do bot pronto (criado no @BotFather). O onboard vai pedir.

Conectar ao Telegram

02

O Telegram é o canal principal. É por onde você conversa com seu agente.

1. **Criar bot no @BotFather**
   Abrir @BotFather no Telegram, enviar /newbot, definir nome e username. Copiar o token gerado.

2. **Configurar o token no OpenClaw**
   O onboard pede o token. Se já fez onboard, configurar manualmente:

```
openclaw config set telegram.token "SEU_TOKEN_AQUI"
```

3. **Configurar allowlist (CRÍTICO)**
   Seu Telegram ID na allowlist. Sem isso, qualquer pessoa comanda o agente. Pegar seu ID: enviar mensagem para @userinfobot.

```
openclaw config set telegram.dmPolicy allowlist
openclaw config set telegram.dmAllowlist -- "[SEU_TELEGRAM_ID]"
```

4. **Iniciar o gateway**
   Subir o agente. Primeira mensagem no Telegram e ele responde.

```
openclaw gateway start
```

5. **Instalar como serviço (opcional mas recomendado)**
   Para que o agente reinicie automaticamente se o servidor reiniciar.

```
openclaw gateway install
```

Lição real: Deixamos dmPolicy como "open" nos primeiros 5 minutos. Qualquer pessoa que encontrasse o bot poderia comandar o agente. Corrigimos imediatamente. Allowlist primeiro, sempre.

Criar a personalidade

03

Um agente sem identidade é um ChatGPT caro. Com identidade, ele vira parte do time.

1. **Editar SOUL.md**
   O arquivo mais importante. Define quem o agente é: tom de voz, comportamento, limites, proatividade. Quanto mais detalhado, melhor. Anti-patterns com exemplos. "Never dos" explícitos. O nosso tem 80+ linhas.

2. **Editar IDENTITY.md**
   Nome, email, papel, nível de autonomia. Identidade formal do agente no sistema.

3. **Editar USER.md**
   Quem é você. Preferências, horários, regras de comunicação. O agente lê isso toda sessão e sabe como te tratar. O nosso tem informações de negócio, equipe, tom de voz para conteúdo.

4. **Editar TOOLS.md**
   Notas sobre ferramentas, APIs, paths do sistema. Cheat sheet do agente.

5. **Reiniciar o gateway**
   O gateway cacheia esses arquivos. Toda edição exige restart para aplicar.

```
openclaw gateway restart
```

Todos os arquivos ficam em: ~/.openclaw/workspace/
Editar = mudar comportamento. Markdown puro. Sem código. Sem deploy.

Blindar o sistema

04

Segurança não é etapa final. É a primeira coisa depois de ligar.

1. **dmPolicy: allowlist**
   Já feito na Fase 2. Confirmar que está ativo. Sem isso, nada mais importa.

2. **Instalar 1Password CLI**
   Gerenciador de credenciais. Todas as senhas, tokens e API keys ficam no vault. Zero hardcode.

```
wget -qO /tmp/op.zip
"https://cache.agilebits.com/dist/1P/op2/pkg/v2.30.3/op_linux_amd64_v2.30.3.zip" && unzip
/tmp/op.zip -d ~/.local/bin/
```

3. **Criar vault dedicado**
   No 1Password, criar vault exclusivo para o agente. Nós usamos "Dona". Criar service account com acesso só a esse vault.

4. **Configurar variáveis de ambiente**
   Tokens de API (Anthropic, OpenAI, etc) ficam em variáveis de ambiente no .service, não em arquivos texto.

5. **Testar acesso**
   Confirmar que o agente consegue ler do vault e que nenhum token está em texto claro.

```
op read "op://Dona/RapidAPI Key/credential"
```

Regras de segurança que seguimos desde o dia 1

- Toda credencial no 1Password. Sem exceções.
- Allowlist antes de qualquer outra configuração.
- Nunca automatizar updates do OpenClaw (incidente de 5h de downtime).
- Audit semanal automatizado via cron.
- Rotação trimestral de API keys.

Criar *memória* permanente

05

Sem memória, o agente começa do zero toda vez. Com memória, ele acumula conhecimento.

1. **Criar MEMORY.md**
   Índice central que referencia todos os topic files. O agente consulta isso para saber onde encontrar cada tipo de informação.

2. **Criar topic files**
   Cada tipo de informação vive em seu arquivo dentro de memory/:

   decisions.md
   Decisões permanentes. O agente nunca esquece o que foi decidido.

   lessons.md
   Erros e acertos. Padrões identificados. Estratégicas (permanentes) e táticas (expiram 30 dias).

   projects.md
   Projetos ativos, status, próximos passos, bloqueios.

   pending.md
   O que está aguardando input. Agente cobra automaticamente.

3. **Indexar a memória**
   Cria índice semântico para buscas por contexto, não por palavra-chave.

   ```
   openclaw memory index --force
   ```

4. **Configurar HEARTBEAT.md**
   Checklist que o agente verifica a cada pulso periódico: pendências antigas, projetos parados, crons falhando, notas para consolidar.

Instalar skills

06

Skills são plugins que dão capacidades ao agente. Sem skills, ele só conversa. Com skills, ele executa.

1. **Ver skills disponíveis**
   OpenClaw vem com 51 skills. Listar quais estão prontas e quais faltam instalar.

```
openclaw skills list
```

2. **Instalar as essenciais**
   As que usamos no dia a dia:

Nano Banana Pro
Geração de fotos via Gemini

Nano PDF
Editar PDFs com linguagem natural

Whisper API
Transcrição de áudio em texto

Video Frames
Extrair frames de vídeos

1Password
Gestão segura de credenciais

Healthcheck
Auditoria de segurança

3. **Configurar APIs externas**
   Algumas skills precisam de API keys: OpenAI (Whisper, DALL-E), Google (Gemini), RapidAPI (Instagram, YouTube, YakPDF). Salvar tudo no 1Password e configurar como variáveis de ambiente.

4. **Instalar binários auxiliares**
   FFmpeg para vídeo/áudio. Puppeteer/Chromium para geração de PDF no servidor.

```
npm i -g puppeteer && npx puppeteer browsers install chrome
```

Configurar *crons* e heartbeats

07

Crons fazem o agente trabalhar sozinho. Heartbeats verificam se está tudo saudável.

1. **Entender a regra de ouro dos crons**
   SEMPRE usar: sessionTarget "isolated" + payload "agentTurn" + delivery "announce". Nunca systemEvent + main (não executa nada, só mostra status ok).

2. **Criar crons essenciais**
   Os que rodamos diariamente:

health-monitor	A cada 1h	Verifica saúde do gateway e APIs
backup-diario	3h	Backup de memória e configurações
checagem-proativa	4x/dia	Pendências, projetos parados, anomalias
org-memória	23h	Consolida notas diárias
security-audit	Semanal	Auditoria completa de segurança

3. **Espaçar crons**
   Mínimo 15-30 minutos entre crons. Colisão de horários causa rate limit e falhas silenciosas.

4. **Usar modelos baratos para automação**
   Heartbeats: Haiku. Crons de execução: Sonnet. Interação direta: Opus. Sem split = US$100+/mês. Com split = US$18-36/mês.

Criar o *time* de agentes

08

Um agente é útil. Dez agentes especializados são um time inteiro.

1. **Criar supergrupo no Telegram com tópicos**
   Um supergrupo com tópicos ativados. Cada tópico = um agente. Adicionar o bot ao grupo e dar admin. Não precisa de múltiplas instâncias.
2. **Criar AGENTS.md**
   Arquivo que mapeia cada topic_id para uma identidade. Quando mensagem chega no tópico 7, o agente assume identidade "Harpia". Tópico 8, assume "Masterson". E assim por diante.
3. **Definir cada agente**
   Para cada agente: nome, papel, tom, reporta para quem, o que faz, o que não faz, base de conhecimento.
4. **Mencionar o sistema de identidades no SOUL.md**
   Se o SOUL.md diz só "Nome: Dona" sem explicar que assumir identidades é parte do design, o modelo resiste a trocar de persona. Isso precisa estar explícito.
5. **Configurar grupo no OpenClaw**
   Adicionar o chat ID do grupo na configuração para o agente responder nos tópicos.

Nosso organograma (10 agentes)

Dona (COO) → Atlas (Estrategista) + Eva (Arquiteta) → Masterson (Escritor), Aurora (Designer), Vega (Social), Harpia (Vídeo), Helios (Tráfego), Orion (Dev), Nexa (Imagem), Doninha (Bot)

Alimentar a base

09

O agente só é tão bom quanto o conhecimento que você dá a ele.

1. **Criar pasta knowledge/**
   Cada agente pode ter sua subpasta com documentos específicos. O agente lê sob demanda via memory_search.

2. **Alimentar com livros e frameworks**
   Nós indexamos 5 livros de copywriting para o Masterson. Cada livro resumido em markdown, salvo em knowledge/masterson/. O agente aplica esses frameworks em tudo que escreve.

3. **Criar Design System**
   Se você produz material visual, documente: cores, fontes, espaçamentos, templates. O nosso (ISIS v7) está salvo e é obrigatório em todos os materiais.

4. **Documentar processos**
   Cada framework operacional vira documento em knowledge/. Framework de tráfego, pipeline de PDF, processo de geração de fotos. Tudo documentado.

5. **Transcrever conteúdo existente**
   Mentorias, cursos, aulas gravadas. Transcrever via Whisper e salvar como base de conhecimento. 100 horas de conteúdo custa ~US$36 para transcrever.

6. **Reindexar após cada adição**
   Toda vez que adicionar conteúdo novo à base:

```
openclaw memory index --force
```

Conectar APIs externas

10

O agente ganha poder real quando conecta a sistemas externos.

1. **Meta Ads API**
   Para análise de tráfego pago. Criar system user no Business Manager, gerar token com permissões ads_read, ads_management, pages_read_engagement. Salvar token como variável de ambiente.

2. **RapidAPI**
   Hub de APIs. Configuramos 5: Instagram (posts e reels), YouTube (vídeos de canal), Whisper via URL, Background Removal, YakPDF (HTML para PDF como fallback).

3. **GitHub (deploy)**
   SSH deploy key para push automático. O agente faz push e dispara redeploy.

4. **Coolify (hosting)**
   Plataforma de deploy. API para disparar redeploy automático após push no GitHub.

5. **Dashboard (Torre Dona)**
   Mission Control conectado ao gateway do OpenClaw. 28 painéis: Tasks, Agents, Crons, Memory, Health, Sessions. Interface visual para monitorar tudo.

Lição real: IPs de cloud (Hetzner, DigitalOcean, AWS) são bloqueados por YouTube, Instagram e X. Usar RapidAPI como proxy para essas plataformas.

10 regras invioláveis

11

Lições de 13 dias rodando agentes de IA em produção.

1. **Crons: isolated + agentTurn + announce**
   Nunca systemEvent + main. Parece funcionar mas não executa nada.
2. **Nunca hardcodar credenciais**
   1Password ou .env com chmod 600. Zero exceções.
3. **dmPolicy: allowlist desde o minuto zero**
   Antes de qualquer outra configuração.
4. **Extrair lições ANTES de compactação**
   Compactação descarta 80% do contexto. Sem extração = perda permanente.
5. **Todo agente começa N1 (Junior)**
   Sem autonomia sem histórico. Output sempre revisado.
6. **Split de modelos por custo**
   Crons: Haiku/Sonnet. Interação: Opus. Economia de 70-80%.
7. **Backup antes de mudanças estruturais**
   config.patch reinicia gateway e mata crons. Criar ROLLBACK.md.
8. **Sub-agent travou: retry 2x, depois alerta**
   Nunca fire and forget. Nunca limbo silencioso.
9. **SOUL.md personalizado é obrigatório**
   SOUL.md genérico = ChatGPT caro.
10. **Creators são skills, não agentes**
    1 agente com 8 skills > 8 agentes separados.

As 10 fases do *zero ao sistema*

Checklist completo. Siga na ordem.

1. **Servidor**
   VPS + Node.js + npm i -g openclaw + openclaw onboard
2. **Canal**
   Bot no Telegram + token + allowlist + gateway start
3. **Identidade**
   SOUL.md + IDENTITY.md + USER.md + TOOLS.md
4. **Segurança**
   1Password + allowlist + variáveis de ambiente
5. **Memória**
   MEMORY.md + topic files + HEARTBEAT.md + indexação
6. **Skills**
   Instalar essenciais + configurar APIs + binários auxiliares
7. **Crons**
   Automações + heartbeats + split de modelos por custo
8. **Multi-agente**
   Supergrupo + tópicos + AGENTS.md + identidades
9. **Conhecimento**
   knowledge/ + livros + design system + transcrições + frameworks
10. **Integrações**
    Meta Ads + RapidAPI + GitHub + Coolify + Dashboard

Isis Moreira · @moreira.isis · Março 2026